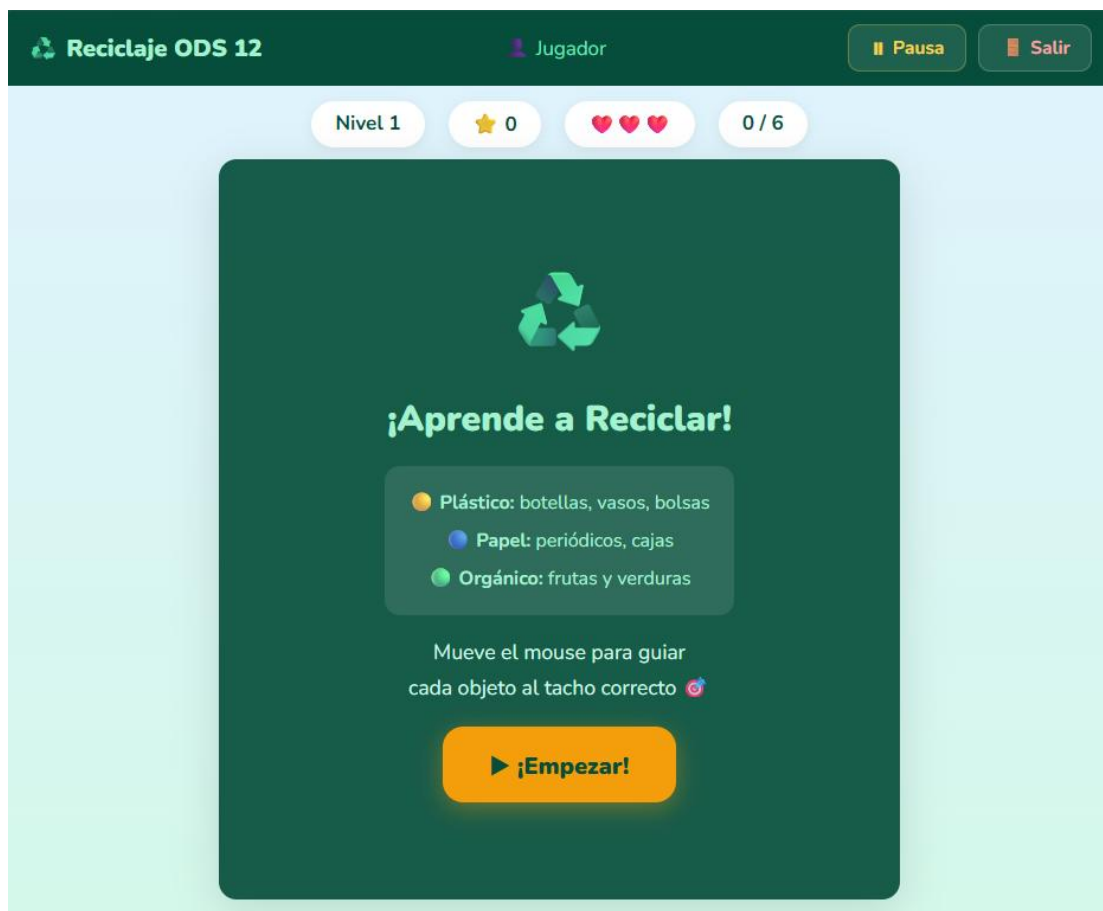


Bitácora

Juego de Reciclaje – ODS 12

Huertas Suárez Daniel Eduardo



Bitácora General del Sistema

El proyecto implementa una arquitectura cliente-servidor desacoplada. El frontend opera con HTML5, Canvas API y JavaScript ES6, mientras que el backend Node.js + Express gestiona la persistencia en SQL Server 2022. El módulo api.js actúa como capa intermediaria que intenta conectar con el backend y, si no está disponible, usa localStorage como respaldo automático.

```
backend > JS database.js > ...
1  const sql = require('mssql');
2  require('dotenv').config();
3
4  const config = {
5    server: 'localhost',
6    database: 'reciclaje_ods12',
7    port: 64456,
8    user: 'ods_user',
9    password: 'Ods12345!',
10   options: {
11     trustServerCertificate: true,
12     enableArithAbort: true
13   }
14 };
15
16 let pool = null;
17
18 async function getPool() {
19   if (pool) return pool;
20   try {
21     pool = await sql.connect(config);
22     console.log('✅ Conexión a SQL Server establecida correctamente');
23     return pool;
24   } catch (error) {
25     console.error('❌ Error al conectar con SQL Server:', error.message);
26     throw error;
27   }
28 }
```

Tabla 1
Responsabilidad de cada módulo del proyecto

Archivo	Capa	Responsabilidad principal	Depende de
index.html	Estructura	Página de login y registro. Redirige a juego.html tras autenticación exitosa.	estilos.css · db.js · api.js · auth.js
juego.html	Estructura	Página del juego: canvas, HUD, botón pausa y cinco pantallas superpuestas (inicio, nivel, game over, completo, pausa).	estilos.css · db.js · api.js · juego.js
estilos.css	Presentación	Paleta ODS 12, fuente Nunito, formularios, HUD, pantallas superpuestas y aviso de rotación móvil.	Ninguna.
db.js	Datos (local)	Base de datos localStorage: registrar, login, guardar progreso, obtener jugador. Actúa como fallback cuando el backend no está disponible.	Ninguna.
api.js	Comunicación	Capa de comunicación con el backend Node.js via fetch(). Detecta si el servidor está disponible con un ping de 2 segundos. Si no responde, delega automáticamente a db.js.	db.js · backend Node.js

Archivo	Capa	Responsabilidad principal	Depende de
auth.js	Comportamiento	Valida formularios, llama api_registrar() / api_login(), guarda sesión en sessionStorage y redirige al juego.	api.js · db.js
juego.js	Comportamiento	Bucle Canvas, detección de colisiones, sistema de pausa, guardado de progreso via api_guardarProgreso() y carga de nivel desde api_obtenerJugador().	api.js · db.js

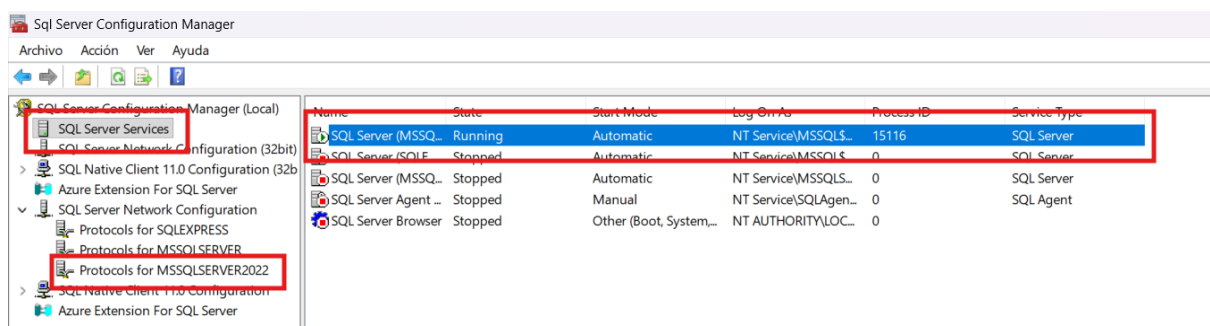
```

7 // -----
8 // 1. CONFIGURACIÓN DE NIVELES ← editar aquí
9 // -----
10 const NIVELES = {
11   1: { velocidad: 1.2, objetivo: 6, nombre: 'Principiante 🌱' },
12   2: { velocidad: 2.0, objetivo: 9, nombre: 'Explorador 🌿' },
13   3: { velocidad: 2.8, objetivo: 12, nombre: 'Guardián 🌲' },
14   4: { velocidad: 3.6, objetivo: 15, nombre: 'Héroe 🌍' }
15 };
16
17 // -----
18 // 2. OBJETOS QUE CAEN (emoji + tipo de tacho)
19 // -----
20 const OBJETOS = [
21   { emoji: '🍷', nombre: 'Botella', tipo: 'plastico', color: '#fbbf24' },
22   { emoji: '🍺', nombre: 'Vaso', tipo: 'plastico', color: '#fbbf24' },
23   { emoji: '👛', nombre: 'Bolsa', tipo: 'plastico', color: '#fbbf24' },
24   { emoji: '📰', nombre: 'Periódico', tipo: 'papel', color: '#60a5fa' },
25   { emoji: '📦', nombre: 'Caja', tipo: 'papel', color: '#60a5fa' },
26   { emoji: '🍌', nombre: 'Plátano', tipo: 'organico', color: '#34d399' },
27   { emoji: '🍏', nombre: 'Manzana', tipo: 'organico', color: '#34d399' },
28   { emoji: '🥬', nombre: 'Verdura', tipo: 'organico', color: '#34d399' }
29 ];
30
31 // -----

```

Sistema de Base de Datos (SQL Server 2022)

El backend implementa persistencia real mediante SQL Server 2022 con autenticación SQL Server. La conexión se gestiona a través del driver mssql en Node.js usando un pool de conexiones. El frontend se comunica con el backend mediante peticiones fetch() asíncronas al servidor Express corriendo en <http://localhost:3000>.



Esquema de la base de datos reciclaje_ods12

```
7  -- Crear base de datos
8  IF NOT EXISTS (SELECT name FROM sys.databases WHERE name = 'reciclaje_ods12')
9  CREATE DATABASE reciclaje_ods12;
10 GO
11
12 USE reciclaje_ods12;
13 GO
14
15 -- Tabla de usuarios
16 IF NOT EXISTS (SELECT * FROM sysobjects WHERE name='usuarios' AND xtype='U')
17 CREATE TABLE usuarios (
18     id                INT IDENTITY(1,1) PRIMARY KEY,
19     nombre            NVARCHAR(100) NOT NULL,
20     usuario           NVARCHAR(50) NOT NULL UNIQUE,
21     password          NVARCHAR(255) NOT NULL,
22     nivel             INT           DEFAULT 1,
23     puntos            INT           DEFAULT 0,
24     estrellas         INT           DEFAULT 0,
25     fecha_registro    DATETIME      DEFAULT GETDATE(),
26     ultima_sesion     DATETIME      NULL
27 );
28 GO
29
```

Tabla 2

Tabla usuarios — estructura de campos

Campo	Tipo SQL	Descripción
id	INT IDENTITY	Clave primaria autoincremental. Generada automáticamente por SQL Server.
nombre	NVARCHAR(100)	Nombre real del jugador ingresado en el formulario de registro.
usuario	NVARCHAR(50)	Nombre de usuario único. Tiene restricción UNIQUE que impide duplicados.
password	NVARCHAR(255)	Contraseña en texto plano. En producción se aplicaría hashing bcrypt. Para el alcance educativo es aceptable.
nivel	INT DEFAULT 1	Último nivel desbloqueado. Solo se actualiza si el nuevo nivel supera el guardado, evitando retrocesos.
puntos	INT DEFAULT 0	Puntaje máximo alcanzado. Solo se actualiza si el nuevo puntaje supera el registro anterior.
estrellas	INT DEFAULT 0	Total de aciertos en la última sesión completada. Se sobrescribe en cada guardado.
fecha_registro	DATETIME	Fecha y hora de creación de la cuenta. Valor por defecto: GETDATE().
ultima_sesion	DATETIME NULL	Fecha y hora del último inicio de sesión exitoso. Se actualiza con GETDATE() en cada login.

```

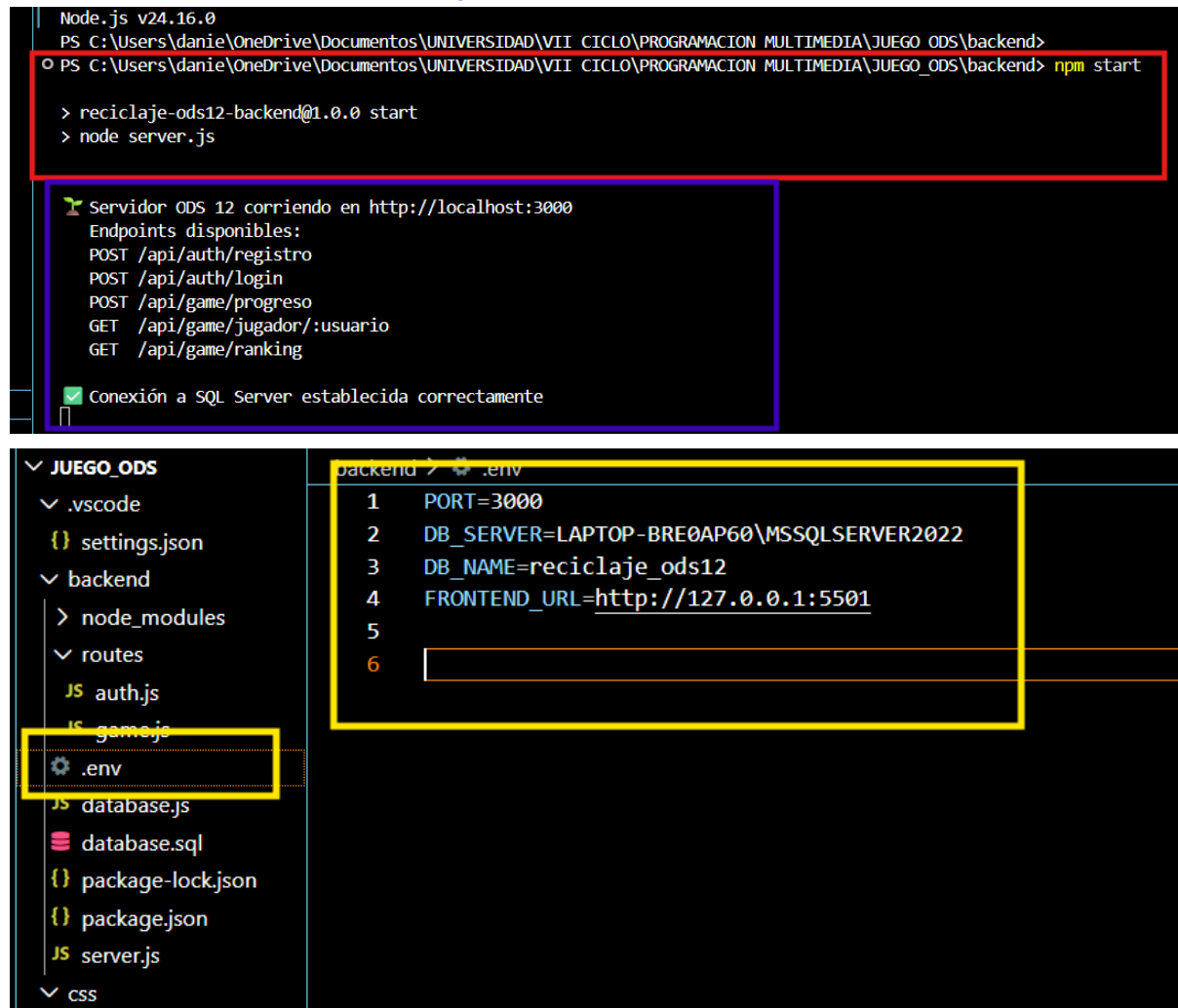
30  -- Tabla de sesiones
31  IF NOT EXISTS (SELECT * FROM sysobjects WHERE name='sesiones' AND xtype='U')
32  CREATE TABLE sesiones (
33      id INT IDENTITY(1,1) PRIMARY KEY,
34      usuario_id INT NOT NULL,
35      nivel_alcanzado INT NOT NULL,
36      puntaje INT NOT NULL,
37      aciertos INT DEFAULT 0,
38      tiempo_segundos INT DEFAULT 0,
39      fecha DATETIME DEFAULT GETDATE(),
40      FOREIGN KEY (usuario_id) REFERENCES usuarios(id)
41  );
42  GO
43
44  -- Vista ranking global
45  IF EXISTS (SELECT * FROM sys.views WHERE name = 'ranking_global')
46  DROP VIEW ranking_global;
47  GO
48  CREATE VIEW ranking_global AS
49  SELECT TOP 10
50      u.usuario,
51      u.nombre,
52      u.puntos AS mejor_puntaje,
53      u.nivel AS nivel_maximo,
54      u.ultima_sesion
55  FROM usuarios u
56  ORDER BY u.puntos DESC;
57  GO

```

Tabla 3
Tabla sesiones — historial de partidas

Campo	Tipo SQL	Descripción
id	INT IDENTITY	Clave primaria autoincremental.
usuario_id	INT (FK)	Clave foránea que referencia usuarios(id). Registra qué jugador generó la sesión.
nivel_alcanzado	INT	Nivel en el que se encontraba el jugador al guardar (game over, pausa o nivel completado).
puntaje	INT	Puntaje acumulado en esa sesión específica.
aciertos	INT DEFAULT 0	Cantidad de objetos clasificados correctamente en esa sesión.
tiempo_segundos	INT DEFAULT 0	Duración de la sesión en segundos, calculado con Date.now() en el frontend.
fecha	DATETIME	Fecha y hora de registro de la sesión. Valor por defecto: GETDATE().

Endpoints de la API REST (Node.js + Express)



```
Node.js v24.16.0
PS C:\Users\danie\OneDrive\Documentos\UNIVERSIDAD\VII CICLO\PROGRAMACION MULTIMEDIA\JUEGO_ODS\backend>
PS C:\Users\danie\OneDrive\Documentos\UNIVERSIDAD\VII CICLO\PROGRAMACION MULTIMEDIA\JUEGO_ODS\backend> npm start

> reciclaje-ods12-backend@1.0.0 start
> node server.js

Servidor ODS 12 corriendo en http://localhost:3000
Endpoints disponibles:
POST /api/auth/registro
POST /api/auth/login
POST /api/game/progreso
GET /api/game/jugador/:usuario
GET /api/game/ranking

Conexión a SQL Server establecida correctamente
```

File Explorer: JUEGO_ODS > backend > .env

```
1 PORT=3000
2 DB_SERVER=LAPTOP-BRE0AP60\MSSQLSERVER2022
3 DB_NAME=reciclaje_ods12
4 FRONTEND_URL=http://127.0.0.1:5501
5
6
```

Tabla 4
Endpoints disponibles en <http://localhost:3000>

Método	Ruta	Descripción
GET	/api/ping	Verifica que el servidor está activo. El frontend lo usa para decidir si usa MySQL o localStorage.
POST	/api/auth/registro	Registra un nuevo usuario. Valida campos, verifica que el usuario no exista y lo inserta con sentencias preparadas.
POST	/api/auth/login	Autentica al usuario. Retorna sus datos (nivel, puntos) y actualiza ultima_sesion en SQL Server.
POST	/api/game/progreso	Guarda el progreso: actualiza nivel y puntos en usuarios e inserta un registro en la tabla sesiones con el tiempo jugado.
GET	/api/game/jugador/:usuario	Obtiene los datos actuales del jugador para cargar su nivel al iniciar sesión.

Método	Ruta	Descripción
GET	/api/game/ranking	Devuelve el Top 10 de jugadores ordenados por puntaje descendente para mostrar en la pantalla de juego completo.

Consultas SQL útiles para verificar datos

Las siguientes consultas pueden ejecutarse directamente en SQL Server Management Studio (SSMS):

- Ver todos los usuarios registrados: `SELECT * FROM usuarios;`
- Ver sesiones de un jugador específico: `SELECT * FROM sesiones WHERE usuario_id = (SELECT id FROM usuarios WHERE usuario = 'Angel07');`
- Ver ranking global: `SELECT TOP 10 usuario, nombre, puntos, nivel FROM usuarios ORDER BY puntos DESC;`
- Contar total de partidas jugadas: `SELECT COUNT(*) AS total_sesiones FROM sesiones;`
- Ver último jugador registrado: `SELECT TOP 1 * FROM usuarios ORDER BY fecha_registro DESC;`
- Borrar todos los datos de prueba: `DELETE FROM sesiones; DELETE FROM usuarios;`

The screenshot shows the SQL Server Management Studio (SSMS) interface. The top pane displays a SQL query: `USE reciclaje_ods12; SELECT * FROM usuarios;`. The bottom pane shows the results of the query in a table format. The table has 10 columns: id, nombre, usuario, password, nivel, puntos, estrellas, fecha_registro, and ultima_sesion. There are 3 rows of data.

	id	nombre	usuario	password	nivel	puntos	estrellas	fecha_registro	ultima_sesion
1	1	Angel	Cristiano	0609	1	0	0	2026-06-10 18:08:08.557	NULL
2	2	Danie	Angel07	0609	1	0	0	2026-06-10 18:08:50.443	NULL
3	3	Daniel	Daniel	1234	1	10	1	2026-06-10 18:09:16.947	2026-06-10 18:09:26.650

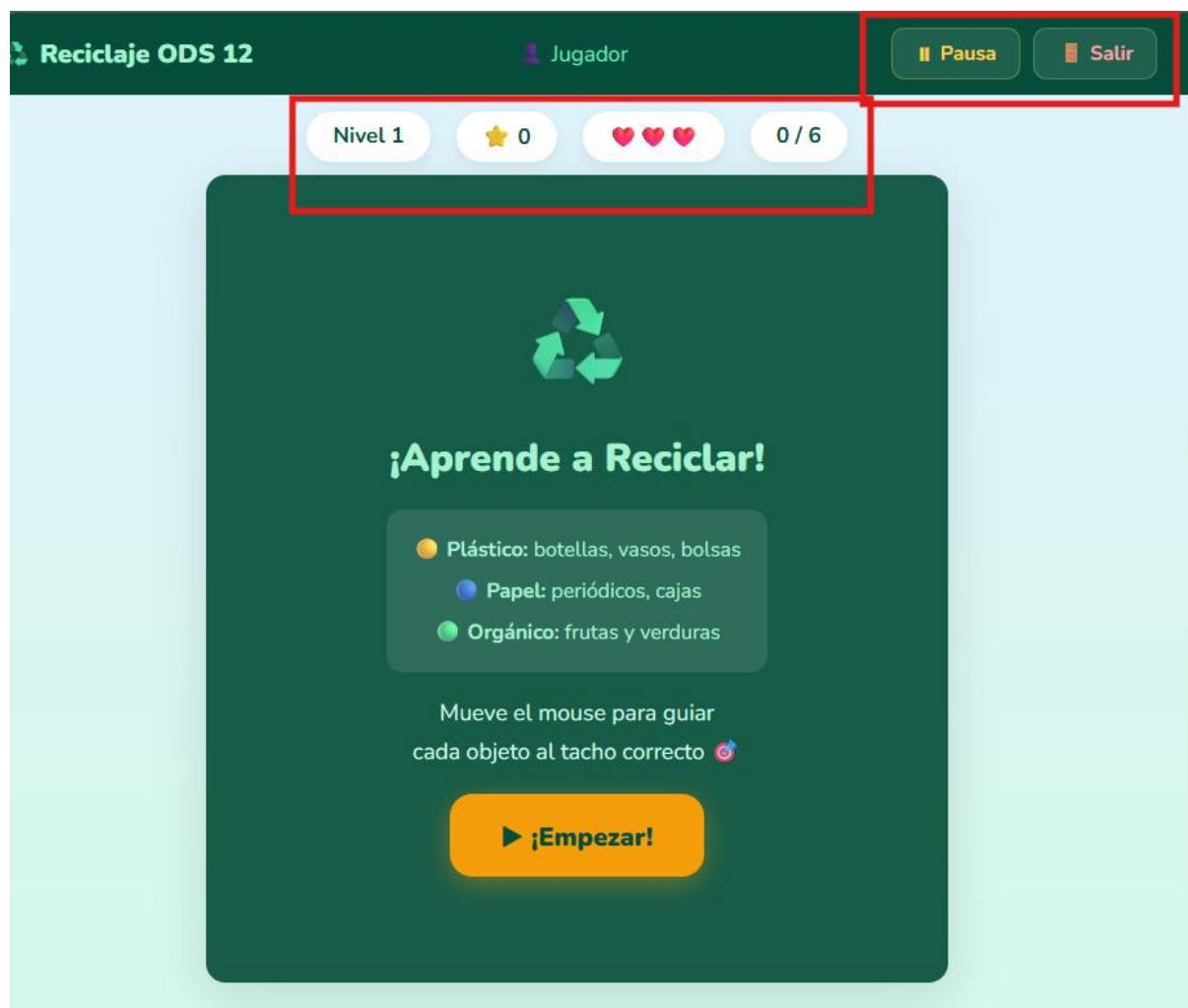
Configuración de Niveles y Objetos

Las configuraciones de niveles y objetos se encuentran al inicio de juego.js bajo las constantes NIVELES y OBJETOS, facilitando su modificación en clase.

Tabla 5

Sistema de niveles del juego

N°	Nombre	Velocidad	Aciertos	Dificultad
1	Principiante 🌱	1.2 px/frame	6	Introductorio. Caída lenta, tiempo de reacción amplio.
2	Explorador 🍃	2.0 px/frame	9	Velocidad moderada. El jugador ya reconoce los tipos de residuo.
3	Guardián 🌲	2.8 px/frame	12	Velocidad elevada. Requiere atención sostenida.
4	Héroe 🌍	3.6 px/frame	15	Máxima dificultad. Desafío para jugadores experimentados.



Glosario de Términos Técnicos

Definiciones operativas de los conceptos empleados en el proyecto.

Tabla 6

Glosario de términos técnicos

Término	Definición operativa
Canvas API	Interfaz HTML5 para dibujar gráficos 2D sobre el elemento <canvas> mediante JavaScript. Todo el juego se renderiza en el canvas fotograma a fotograma.
ctx.clearRect()	Borra un área rectangular del lienzo. Se invoca al inicio de cada fotograma para eliminar el dibujo anterior. Sin este paso los objetos dejarían rastros visuales.
requestAnimationFrame()	Función nativa que programa la ejecución de un callback antes del próximo repintado de pantalla (~60 fps). Produce la animación fluida del juego.
Bucle de animación	Patrón donde una función se invoca fotograma a fotograma: limpiar canvas → dibujar fondo → dibujar tachos → dibujar objeto → mover → verificar colisión → requestAnimationFrame.
SQL Server 2022	Motor de base de datos relacional de Microsoft usado para persistir usuarios y sesiones del juego. Se accede mediante el driver mssql en Node.js con sentencias preparadas.
mssql	Driver Node.js para conectarse a SQL Server. Usa un pool de conexiones para manejar múltiples peticiones simultáneas de forma eficiente.
Sentencias preparadas	Consultas SQL parametrizadas que separan el código SQL de los datos del usuario, previniendo inyección SQL. Ejemplo: .input('usuario', sql.VarChar, usuario).
Express.js	Framework minimalista de Node.js para crear servidores HTTP. En este proyecto gestiona las rutas de la API REST (/api/auth y /api/game).
API REST	Arquitectura de comunicación cliente-servidor basada en HTTP. El frontend llama endpoints con fetch() y el backend responde en formato JSON.
fetch()	API JavaScript para hacer peticiones HTTP asíncronas. Se usa en api.js para comunicarse con el backend Node.js y guardar/obtener datos de SQL Server.
Pool de conexiones	Conjunto de conexiones a la base de datos mantenidas abiertas y reutilizables. Evita el costo de abrir una nueva conexión en cada petición.
localStorage	API del navegador para almacenamiento local persistente. Actúa como sistema de respaldo (fallback) cuando el backend Node.js no está disponible.

Término	Definición operativa
sessionStorage	API similar a localStorage pero de duración de sesión. Guarda el usuario activo ('jugador_activo') para identificar quién está jugando.
CORS	Política de seguridad del navegador que controla qué dominios pueden hacer peticiones a un servidor. Configurado en Express para aceptar peticiones desde el frontend.
async/await	Sintaxis JavaScript para manejar operaciones asíncronas (peticiones de red, base de datos) de forma legible. Todas las funciones que llaman al backend usan esta sintaxis.
Pausa / Reanuda	Cuando pausado = true el bucle retorna sin pedir el siguiente fotograma, deteniendo la animación. reanudarJuego() llama requestAnimationFrame(bucle) para reiniciarlo.
ODS 12	Objetivo de Desarrollo Sostenible número 12 de la ONU: Producción y Consumo Responsables. El juego educa sobre clasificación de residuos sólidos desde la infancia.
Separación de incumbencias	Principio de arquitectura: cada módulo tiene una sola responsabilidad. HTML = estructura, CSS = presentación, db.js = datos locales, api.js = comunicación, juego.js = comportamiento.

Sistema de Pausa y Guardado Automático

Tabla 7

Opciones del menú de pausa

Opción	Acción técnica	Resultado
▶ Continuar jugando	pausado = false → requestAnimationFrame(bucle)	El juego reanuda desde el estado exacto en que se pausó.
🔄 Reiniciar nivel	iniciarNivel(): vidas=3, aciertos=0, velocidad inicial	Comienza el nivel actual desde cero sin afectar el progreso guardado.
📄 Guardar y salir	api_guardarProgreso() → sessionStorage.remove → index.html	Guarda en SQL Server (y localStorage), cierra sesión y regresa al login.

```

41 <canvas id="gameCanvas" width="460" height="500"></canvas>
42
43 <!-- Pantalla: Inicio del nivel -->
44 <div class="pantalla" id="pantalla-inicio">
45   <div class="emoji-grande">🌱</div>
46   <h2>¡Aprende a Reciclar!</h2>
47   <div class="pistas">
48     <strong>Plástico:</strong> botellas, vasos, bolsas<br>
49     <strong>Papel:</strong> periódicos, cajas<br>
50     <strong>Orgánico:</strong> frutas y verduras
51   </div>
52   <p>Mueve el mouse para guiar<br>cada objeto al tachito correcto 🎯</p>
53   <button class="btn-juego" onclick="btnIniciar()">▶ ¡Empezar!</button>
54 </div>
55
56 <!-- Pantalla: Nivel completado -->
57 <div class="pantalla" id="pantalla-nivel">
58   <div class="emoji-grande">🎉</div>
59   <h2 id="nivel-completo-num">¡Nivel completado!</h2>
60   <p id="nivel-siguiente-num">Siguiendo nivel...</p>
61   <button class="btn-juego" onclick="btnSiguienteNivel()">▶ Siguiendo nivel</button>
62 </div>
63

```

INDICACIONES

Para poder tener el juego completo se subiera a GitHub y las indicaciones estarán en un archivo `indicaciones.md`

JUEGO_ODS

> .vscode

> backend

> css

> javascript

BITACORA.docx

BITACORA.pdf

index.html

indicaciones.md

juego.html

Producción y Consumo Responsable

Autor

HUERTAS SUÁREZ DANIEL EDUARDO

Juego educativo interactivo desarrollado con HTML5 Canvas, JavaScript ES6, Node.js y SQL Server 2022. Diseñado para niños en proceso de formación sobre clasificación de residuos sólidos.

Estructura del Proyecto

JUEGO_ODS/

├── index.html

├── juego.html

├── css/

│ ├── estilos.css

├── javascript/

│ ├── db.js

│ ├── api.js

│ ├── auth.js

│ └── juego.js

└── backend/

├── server.js

├── database.js

├── database.sql

├── package.json

├── .env

└── routes/

├── auth.js

└── game.js

├── Pantalla de login y registro

├── Pantalla del juego

├── Estilos de toda la aplicación

├── Base de datos localStorage (fallback)

├── Comunicación con el backend

├── Lógica de login y registro

├── Lógica del juego y Canvas

├── Servidor Node.js + Express

├── Conexión a SQL Server

├── Script de creación de tablas

├── Dependencias Node.js

├── Variables de entorno (no subir a GitHub)

├── Rutas de autenticación

└── Rutas del juego y ranking

🎮 ¿Cómo se juega?

1. Regístrate con tu nombre y usuario

2. Inicia sesión para entrar al juego

3. Mueve el mouse para guiar el objeto que cae

4. Llévelo al tacho correcto antes de que toque el suelo:

○ Plástico → botellas, vasos, bolsas

○ Papel → periódicos, cajas

○ Orgánico → frutas y verduras

5. Cada acierto suma puntos y aumenta la velocidad

6. Tienes **3 vidas** — si pierdes todas, es Game Over

7. Al completar el objetivo de aciertos, subes de nivel

Niveles

Nivel	Nombre	Velocidad	Objetivo
1	Principiante 🌱	Lenta	6 aciertos
2	Explorador 🌿	Media	9 aciertos
3	Guardián 🍀	Rápida	12 aciertos
4	Héroe 🌟	Muy rápida	15 aciertos

⚙️ Requisitos Previos

Antes de instalar, asegúrate de tener:

- Node.js v18 o superior
- SQL Server 2022 (Developer o Express Edition)
- SQL Server Management Studio (SSMS)
- Editor de código: VS Code con extensión Live Server